



Stochastic Processes

Professor: Dr. Safari

Mehrnaz Yazdani, 610301201

Saba Naji, 610301190

Part 1:

- a) The Standardized Precipitation Index (SPI) is used to indicate drought or wet conditions. If the SPI is negative, it indicates drought conditions, and is represented by 0. If the SPI is positive, it indicates wet conditions, and is represented by 1.

```
def SPI(ds, thresh):
    ds_ma = ds.rolling(thresh, center=False).mean()
    ds_ma[ds_ma <= 0] = np.nan
    ds_In = np.log(ds_ma)
    ds_In[np.isinf(ds_In)] = np.nan
    ds_mu = np.nanmean(ds_ma)
    ds_sum = np.nansum(ds_In)
    n = len(ds_In[thresh-1:])
    A = np.log(ds_mu) - (ds_sum / n)
    alpha = (1 / (4 * A)) * (1 + (1 + ((4 * A) / 3))**0.5)
    beta = ds_mu / alpha
    gamma = st.gamma.cdf(ds_ma, a=alpha, scale=beta)
    norm_spi = st.norm.ppf(gamma, loc=0, scale=1)
    return norm_spi
```

This function SPI calculates the Standardized Precipitation Index (SPI) for a series of precipitation data.

Moving Average: First, it computes the moving average of precipitation data using a window size specified by thresh.

Handling Invalid Values: Negative values are set to NaN, and the natural logarithm of the moving average is taken.

Statistical Calculations: Parameters for the gamma distribution (α and β) are calculated, and the gamma cumulative distribution function is applied to the moving average data.

Normalization: Finally, a standard normal distribution is applied to the gamma distribution to obtain the SPI.

```

file_path = 'project.xlsx'
workbook = openpyxl.load_workbook(file_path)
sheet = workbook.active

data = []
for row_number in range(2, sheet.max_row + 1):
    date_cell = sheet.cell(row=row_number, column=1).value
    rain_cell = sheet.cell(row=row_number, column=6).value
    if date_cell and rain_cell:
        date = pd.to_datetime(date_cell)
        rain_value = float(rain_cell.replace(' mm', ''))
        data.append({'Date': date, 'Rain': rain_value})

df = pd.DataFrame(data)
df.set_index('Date', inplace=True)

yearly_mean_rain = df.resample('Y').mean()
spi_values_yearly = SPI(yearly_mean_rain['Rain'], 1)
spi_df_yearly = pd.DataFrame(spi_values_yearly, index=yearly_mean_rain.index, columns=['SPI'])
spi_df_yearly['SPI_binary'] = np.where(spi_df_yearly['SPI'] <= 0, 0, 1)
SPI_binary_yearly = spi_df_yearly['SPI_binary'].tolist()

monthly_mean_rain = df.resample('M').mean()
spi_values_monthly = SPI(monthly_mean_rain['Rain'], 1)
spi_df_monthly = pd.DataFrame(spi_values_monthly, index=monthly_mean_rain.index, columns=['SPI'])
spi_df_monthly['SPI_binary'] = np.where(spi_df_monthly['SPI'] <= 0, 0, 1)
SPI_binary_monthly = spi_df_monthly['SPI_binary'].tolist()
spi_df_monthly['Mean_Rainfall'] = monthly_mean_rain['Rain'].values

```

Load Data: It loads an Excel file named 'project.xlsx' using openpyxl and selects the active sheet.

Extract and Clean Data: It reads the data from columns for date and precipitation, converts the date to a datetime object, and cleans the precipitation values by removing the unit ' mm'. The data is then stored in a list of dictionaries.

Create DataFrame: The list is converted into a Pandas DataFrame, with the date set as the index.

Calculate Yearly SPI:

- **Resample Data:** Computes the yearly average rainfall.
- **Compute SPI:** Calculates the SPI values for yearly data using the SPI function.
- **Binary SPI:** Converts SPI values to binary, where 0 indicates drought and 1 indicates wet conditions.

Calculate Monthly SPI:

- **Resample Data:** Computes the monthly average rainfall.
- **Compute SPI:** Calculates the SPI values for monthly data using the SPI function.
- **Binary SPI:** Converts SPI values to binary.
- **Add Mean Rainfall:** Adds the mean monthly rainfall to the DataFrame.

Store Results: The binary SPI values for both yearly and monthly data are converted to lists for further use.

The binary SPI numbers for each month are saved in an Excel file as shown below:

Date	SPI	SPI_binary
01/31/2009	-0.315847894	0
02/28/2009	-0.245733629	0
03/31/2009	-1.059901378	0
04/30/2009	0.811583546	1
05/31/2009	-0.061226081	0
06/30/2009	0.346031566	1
07/31/2009		1
08/31/2009	-2.541787097	0
09/30/2009	-0.687867724	0
10/31/2009	-0.858588762	0
11/30/2009	0.295233225	1
12/31/2009	-0.3008661	0
01/31/2010	-1.025222069	0
02/28/2010	-0.141321054	0
03/31/2010	-0.043347316	0
04/30/2010	-0.196285353	0
05/31/2010	0.042281284	1
06/30/2010	-1.247299838	0
07/31/2010	-2.149624214	0
08/31/2010	0.328214682	1
09/30/2010	0.399461261	1
10/31/2010	-0.17465781	0
11/30/2010	-0.378293861	0
12/31/2010	-1.115247999	0
01/31/2011	-0.3008661	0
02/28/2011	-0.230201011	0
03/31/2011	0.599467699	1
04/30/2011	-0.109019709	0
05/31/2011	0.54280817	1
06/30/2011	0.611173976	1
07/31/2011	-0.091637765	0
08/31/2011	-0.168052493	0
09/30/2011	-1.059309697	0
10/31/2011	1.086560891	1
11/30/2011	0.403826155	1
12/31/2011	-1.042356636	0
01/31/2012	-0.208282318	0
02/29/2012	-0.452539693	0
03/31/2012	0.173149771	1
04/30/2012	1.135228227	1
05/31/2012	1.250972575	1
06/30/2012	0.913207901	1
07/31/2012	0.569503582	1
08/31/2012	-0.470270621	0
09/30/2012	-1.077880275	0
10/31/2012	-0.014129969	0
11/30/2012	0.119799362	1
12/31/2012	-0.60202421	0
01/31/2013	-0.427279487	0

02/28/2013	-0.654031218	0
03/31/2013	-0.079378605	0
04/30/2013	-0.21742357	0
05/31/2013	0.538956944	1
06/30/2013	-0.455730331	0
07/31/2013	-2.300273479	0
08/31/2013	-1.115247999	0
09/30/2013	-0.989401996	0
10/31/2013	-0.622468498	0
11/30/2013	0.109035296	1
12/31/2013	0.053260913	1
01/31/2014	-0.709271062	0
02/28/2014	-0.690335194	0
03/31/2014	0.106766831	1
04/30/2014	0.717956458	1
05/31/2014	0.906823472	1
06/30/2014	0.372993519	1
07/31/2014	-0.23593733	0
08/31/2014	0.565717723	1
09/30/2014	-1.023530309	0
10/31/2014	-0.622468498	0
11/30/2014	-0.337702023	0
12/31/2014	-0.346470167	0
01/31/2015	-0.622468498	0
02/28/2015	0.987253257	1
03/31/2015	0.491975896	1
04/30/2015	0.521116859	1
05/31/2015	0.54280817	1
06/30/2015	-0.040000099	0
07/31/2015	-0.037446761	0
08/31/2015	-0.3941784	0
09/30/2015	0.266727999	1
10/31/2015	0.546649874	1
11/30/2015	0.346031566	1
12/31/2015	0.503856801	1
01/31/2016	0.376292703	1
02/29/2016	0.024429687	1
03/31/2016	0.758409702	1
04/30/2016	0.446777618	1
05/31/2016	0.212396103	1
06/30/2016	0.346031566	1
07/31/2016	-0.612188813	0
08/31/2016	-1.52752111	0
09/30/2016	-0.824746691	0
10/31/2016	-1.264586907	0
11/30/2016	-0.046110901	0
12/31/2016	1.319702309	1
01/31/2017	0.401856526	1
02/28/2017	0.042871937	1
03/31/2017	0.535096133	1

04/30/2017	-0.184515823	0
05/31/2017	0.37159124	1
06/30/2017	-0.067315089	0
07/31/2017	-0.122153668	0
08/31/2017	0.429661037	1
09/30/2017	0.681141089	1
10/31/2017	0.559476029	1
11/30/2017	0.142489251	1
12/31/2017	0.435930828	1
01/31/2018	-0.223915249	0
02/28/2018	-0.009632673	0
03/31/2018	-0.258514164	0
04/30/2018	-0.001046251	0
05/31/2018	0.074613863	1
06/30/2018	-0.408740522	0
07/31/2018	-0.507158341	0
08/31/2018	-0.082139571	0
09/30/2018	-0.061730222	0
10/31/2018	0.299780407	1
11/30/2018	0.181235544	1
12/31/2018	0.239759881	1
01/31/2019	-0.251809773	0
02/28/2019	-0.042282444	0
03/31/2019	-0.008615258	0
04/30/2019	-0.12322416	0
05/31/2019	-0.016355553	0
06/30/2019	0.291563582	1
07/31/2019	0.43788504	1
08/31/2019	0.21242554	1
09/30/2019	0.432162306	1
10/31/2019	0.404349869	1
11/30/2019	0.292438507	1
12/31/2019	0.249149188	1
01/31/2020	0.177148059	1
02/29/2020	-0.026383501	0
03/31/2020	-0.344796576	0
04/30/2020	-0.449014614	0
05/31/2020	-0.306630533	0
06/30/2020	-0.137091546	0
07/31/2020	-0.027341757	0
08/31/2020	0.178452952	1
09/30/2020	0.091635897	1
10/31/2020	-0.043797231	0
11/30/2020	-0.151005623	0
12/31/2020	-0.083788029	0

The binary SPI numbers for each year are saved in an Excel file as shown below:

Date	SPI	SPI_binary
12/31/2009	-0.963114806	0
12/31/2010	-1.033766507	0
12/31/2011	-0.162390282	0
12/31/2012	0.079327778	1
12/31/2013	-1.124512624	0
12/31/2014	-0.414378718	0
12/31/2015	0.131468114	1
12/31/2016	-0.161544729	0
12/31/2017	-0.571253993	0
12/31/2018	0.276632683	1
12/31/2019	1.323986672	1
12/31/2020	2.508965345	1

- b) Now that we have binary indicators (0 or 1) representing drought or wet conditions for each month and each year, we'll create a transition matrix. This matrix will illustrate the changes in conditions between months and years.

```
def generate_all_patterns(pattern_length):  
    num_patterns = 2**pattern_length  
    all_patterns = []  
    for i in range(num_patterns):  
        pattern = []  
        for j in range(pattern_length):  
            pattern.append((i >> (pattern_length - 1 - j)) & 1)  
        all_patterns.append(pattern)  
    return all_patterns
```

This function generates all possible binary patterns of a specified length.

num_patterns: Calculates the total number of binary patterns by raising 2 to the power of pattern_length.

Loop through num_patterns: For each integer from 0 to num_patterns - 1, it constructs a binary pattern.

Construct Pattern: Each pattern is created by extracting individual bits from the integer and arranging them in the correct order.

Return: The function returns a list of all binary patterns of the given length.

```
def count_pattern_occurrences(data, pattern):
    pattern_count = 0
    pattern_with_rain_count = 0
    pattern_length = len(pattern)
    for i in range(len(data) - pattern_length):
        if data[i:i + pattern_length] == pattern:
            pattern_count += 1
            if data[i + pattern_length] == 1:
                pattern_with_rain_count += 1
    return pattern_count, pattern_with_rain_count

def calculate_probabilities(data, pattern_length):
    all_patterns = generate_all_patterns(pattern_length)
    pattern_probabilities = {}
    for pattern in all_patterns:
        pattern_count, rain_count = count_pattern_occurrences(data, pattern)
        total_occurrences = pattern_count
        probability_of_rain = rain_count / total_occurrences if total_occurrences > 0 else 0
        probability_of_no_rain = 1 - probability_of_rain
        pattern_probabilities[tuple(pattern)] = (probability_of_no_rain, probability_of_rain)
    return pattern_probabilities
```

Function 1: count_pattern_occurrences

This function counts how often a specified binary pattern appears in a sequence and how many times it is followed by a 1.

- **Inputs:** data (a list of binary values) and pattern (a binary pattern).
- **Process:** It scans the data for occurrences of the pattern and checks if the value immediately following each occurrence is 1.
- **Outputs:** Returns two counts: the total number of occurrences of the pattern and the number of those occurrences followed by a 1.

Function 2: calculate_probabilities

This function calculates the probabilities of rain following all possible binary patterns of a given length.

- **Inputs:** data (a list of binary values) and pattern_length (length of the binary patterns).
- **Process:** Generates all binary patterns of the specified length, counts their occurrences and the number of times they are followed by a 1, then computes the probability of rain and no rain for each pattern.

- **Outputs:** Returns a dictionary where each key is a binary pattern (as a tuple) and the value is a tuple of probabilities: (probability of no rain, probability of rain).

```
def calculate_log_likelihood(pattern_probabilities, data, pattern_length):
    log_likelihood = 0
    for pattern, (no_rain_prob, rain_prob) in pattern_probabilities.items():
        pattern_count, rain_count = count_pattern_occurrences(data, list(pattern))
        no_rain_count = pattern_count - rain_count

        if no_rain_prob > 0 and no_rain_count > 0:
            log_likelihood += no_rain_count * np.log(no_rain_prob)
        if rain_prob > 0 and rain_count > 0:
            log_likelihood += rain_count * np.log(rain_prob)

    return log_likelihood

def calculate_transition_matrix(data, pattern_length, label):
    pattern_probabilities = calculate_probabilities(data, pattern_length)
    patterns = generate_all_patterns(pattern_length)
    transition_matrix = []
    for pattern in patterns:
        no_rain_prob, rain_prob = pattern_probabilities[tuple(pattern)]
        transition_matrix.append([no_rain_prob, rain_prob])
    transition_df = pd.DataFrame(transition_matrix, index=[tuple(p) for p in patterns], columns=['P(0)', 'P(1)'])

    # Calculate AIC and BIC
    n = len(data)
    k = (2**pattern_length) * (2**pattern_length) - (2**pattern_length)
    log_likelihood = calculate_log_likelihood(pattern_probabilities, data, pattern_length)
    aic = 2 * k - 2 * log_likelihood
    bic = np.log(n) * k - 2 * log_likelihood

    print(f"\n{n}{label} Transition Matrix for Pattern Length {pattern_length}:")
    print(transition_df)
    print(f"AIC: {aic:.2f}")
    print(f"BIC: {bic:.2f}")

    return transition_df, aic, bic
```

Function 1: calculate_log_likelihood

This function computes the log-likelihood of a model given binary pattern probabilities and data.

Inputs: pattern_probabilities (dictionary of probabilities for each pattern), data (binary sequence), and pattern_length (length of the patterns).

Process: For each pattern, it calculates how likely the observed counts of no rain and rain are, based on the model probabilities. It then sums the log-likelihood contributions for all patterns.

Output: Returns the total log-likelihood value.

Function 2: calculate_transition_matrix

This function creates a transition matrix for binary patterns, calculates model fit statistics (AIC and BIC), and prints the results.

Inputs: data (binary sequence), pattern_length (length of the patterns), and label (identifier for the output).

Process:

Computes the probabilities for all patterns.

Constructs a transition matrix where each row represents a pattern with its probabilities of no rain and rain.

Calculates AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) to assess model fit.

Outputs: Prints the transition matrix, AIC, and BIC, and returns the transition matrix along with the AIC and BIC values.

Note: AIC and BIC are used to evaluate the goodness-of-fit for the model and compare different models based on their complexity and fit.

The transition probabilities are based on the state from the previous month:

$$\begin{bmatrix} P(0 \rightarrow 0) & P(0 \rightarrow 1) \\ P(1 \rightarrow 0) & P(1 \rightarrow 1) \end{bmatrix} = \begin{bmatrix} 0.689189 & 0.310811 \\ 0.323529 & 0.676471 \end{bmatrix}$$

The transition probabilities are based on the state from the previous year:

$$\begin{bmatrix} P(0 \rightarrow 0) & P(0 \rightarrow 1) \\ P(1 \rightarrow 0) & P(1 \rightarrow 1) \end{bmatrix} = \begin{bmatrix} 0.571429 & 0.428571 \\ 0.500000 & 0.500000 \end{bmatrix}$$

The transition probabilities are based on the state from two months prior:

$$\begin{bmatrix} P((0,0) \rightarrow (0,0)) & P((0,0) \rightarrow (0,1)) & P((0,0) \rightarrow (1,0)) & P((0,0) \rightarrow (1,1)) \\ P((0,1) \rightarrow (0,0)) & P((0,1) \rightarrow (0,1)) & P((0,1) \rightarrow (1,0)) & P((0,1) \rightarrow (1,1)) \\ P((1,0) \rightarrow (0,0)) & P((1,0) \rightarrow (0,1)) & P((1,0) \rightarrow (1,0)) & P((1,0) \rightarrow (1,1)) \\ P((1,1) \rightarrow (0,0)) & P((1,1) \rightarrow (0,1)) & P((1,1) \rightarrow (1,0)) & P((1,1) \rightarrow (1,1)) \end{bmatrix} = \begin{bmatrix} 0.647059 & 0.352941 & 0 & 0 \\ 0.347826 & 0.652174 & 0 & 0 \\ 0.772727 & 0.227273 & 0 & 0 \\ 0.311111 & 0.688889 & 0 & 0 \end{bmatrix}$$

The transition probabilities are based on the state from two years prior:

$$\begin{bmatrix} P((0,0) \rightarrow (0,0)) & P((0,0) \rightarrow (0,1)) & P((0,0) \rightarrow (1,0)) & P((0,0) \rightarrow (1,1)) \\ P((0,1) \rightarrow (0,0)) & P((0,1) \rightarrow (0,1)) & P((0,1) \rightarrow (1,0)) & P((0,1) \rightarrow (1,1)) \\ P((1,0) \rightarrow (0,0)) & P((1,0) \rightarrow (0,1)) & P((1,0) \rightarrow (1,0)) & P((1,0) \rightarrow (1,1)) \\ P((1,1) \rightarrow (0,0)) & P((1,1) \rightarrow (0,1)) & P((1,1) \rightarrow (1,0)) & P((1,1) \rightarrow (1,1)) \end{bmatrix}$$

The transition probabilities are based on the state from three months prior:

$$\begin{array}{c}
P((0,0,0) \rightarrow (0,0,0)) \quad P((0,0,0) \rightarrow (0,0,1)) \quad P((0,0,0) \rightarrow (0,1,0)) \quad P((0,0,0) \rightarrow (0,1,1)) \quad P((0,0,0) \rightarrow (1,0,0)) \quad P((0,0,0) \rightarrow (1,0,1)) \quad P((0,0,0) \rightarrow (1,1,0)) \quad P((0,0,0) \rightarrow (1,1,1)) \\
P((0,0,1) \rightarrow (0,0,0)) \quad P((0,0,1) \rightarrow (0,0,1)) \quad P((0,0,1) \rightarrow (0,1,0)) \quad P((0,0,1) \rightarrow (0,1,1)) \quad P((0,0,1) \rightarrow (1,0,0)) \quad P((0,0,1) \rightarrow (1,0,1)) \quad P((0,0,1) \rightarrow (1,1,0)) \quad P((0,0,1) \rightarrow (1,1,1)) \\
P((0,1,0) \rightarrow (0,0,0)) \quad P((0,1,0) \rightarrow (0,0,1)) \quad P((0,1,0) \rightarrow (0,1,0)) \quad P((0,1,0) \rightarrow (0,1,1)) \quad P((0,1,0) \rightarrow (1,0,0)) \quad P((0,1,0) \rightarrow (1,0,1)) \quad P((0,1,0) \rightarrow (1,1,0)) \quad P((0,1,0) \rightarrow (1,1,1)) \\
P((0,1,1) \rightarrow (0,0,0)) \quad P((0,1,1) \rightarrow (0,0,1)) \quad P((0,1,1) \rightarrow (0,1,0)) \quad P((0,1,1) \rightarrow (0,1,1)) \quad P((0,1,1) \rightarrow (1,0,0)) \quad P((0,1,1) \rightarrow (1,0,1)) \quad P((0,1,1) \rightarrow (1,1,0)) \quad P((0,1,1) \rightarrow (1,1,1)) \\
P((1,0,0) \rightarrow (0,0,0)) \quad P((1,0,0) \rightarrow (0,0,1)) \quad P((1,0,0) \rightarrow (0,1,0)) \quad P((1,0,0) \rightarrow (0,1,1)) \quad P((1,0,0) \rightarrow (1,0,0)) \quad P((1,0,0) \rightarrow (1,0,1)) \quad P((1,0,0) \rightarrow (1,1,0)) \quad P((1,0,0) \rightarrow (1,1,1)) \\
P((1,0,1) \rightarrow (0,0,0)) \quad P((1,0,1) \rightarrow (0,0,1)) \quad P((1,0,1) \rightarrow (0,1,0)) \quad P((1,0,1) \rightarrow (0,1,1)) \quad P((1,0,1) \rightarrow (1,0,0)) \quad P((1,0,1) \rightarrow (1,0,1)) \quad P((1,0,1) \rightarrow (1,1,0)) \quad P((1,0,1) \rightarrow (1,1,1)) \\
P((1,1,0) \rightarrow (0,0,0)) \quad P((1,1,0) \rightarrow (0,0,1)) \quad P((1,1,0) \rightarrow (0,1,0)) \quad P((1,1,0) \rightarrow (0,1,1)) \quad P((1,1,0) \rightarrow (1,0,0)) \quad P((1,1,0) \rightarrow (1,0,1)) \quad P((1,1,0) \rightarrow (1,1,0)) \quad P((1,1,0) \rightarrow (1,1,1)) \\
P((1,1,1) \rightarrow (0,0,0)) \quad P((1,1,1) \rightarrow (0,0,1)) \quad P((1,1,1) \rightarrow (0,1,0)) \quad P((1,1,1) \rightarrow (0,1,1)) \quad P((1,1,1) \rightarrow (1,0,0)) \quad P((1,1,1) \rightarrow (1,0,1)) \quad P((1,1,1) \rightarrow (1,1,0)) \quad P((1,1,1) \rightarrow (1,1,1))
\end{array}$$

The transition probabilities are based on the state from three years prior:

0	0	0	0	0	0	0	1
0.666667	0	0	0	0.333333	0	0	0
1	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0

The transition probabilities are based on the state from four months prior:

[illegible]

The transition probabilities are based on the state from four years prior:

1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.5	0.5	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0

The transition probabilities are based on the state from five months prior:

0.000000	1.000000
0.300000	0.700000
0.666667	0.333333
0.333333	0.666667
0.750000	0.250000
0.000000	1.000000
0.750000	0.250000
0.000000	1.000000
1.000000	0.000000
0.000000	1.000000
1.000000	0.000000
1.000000	0.000000
0.833333	0.166667
0.000000	1.000000
1.000000	0.000000
0.285714	0.714286
0.700000	0.300000
0.400000	0.600000
1.000000	0.000000
0.500000	0.500000
1.000000	0.000000
1.000000	0.000000
1.000000	0.000000
0.500000	0.500000
0.000000	1.000000
1.000000	0.000000
1.000000	0.000000
1.000000	0.000000
0.714286	0.285714
0.294118	0.705882

The remaining states have a transition probability of 0.

The transition probabilities are based on the state from five years prior:

1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
0.0	1.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
0.5	0.5
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
0.0	1.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0
1.0	0.0

The remaining states have a transition probability of 0.

c)

To determine the best state space (SS) model fits for the dataset, the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) were utilized. Here's an explanation of how these criteria were applied and the results obtained:

Explanation of AIC and BIC:

Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) are statistical measures used to compare different models based on their fit and complexity

AIC (Akaike Information Criterion):

AIC evaluates the quality of a model by balancing its goodness-of-fit with its complexity. Lower AIC values indicate a better model that is less likely to overfit the data.

Formula:

$$AIC = 2k - 2\ln(L)$$

where:

k = Number of parameters in the model

L = Maximum value of the likelihood function for the model

BIC (Bayesian Information Criterion):

BIC also assesses model quality but includes a stronger penalty for model complexity, especially with large datasets. Lower BIC values suggest a better model with a balance between fit and simplicity.

Formula:

$$BIC = \log(n) \cdot k - 2\ln(L)$$

where:

n = Number of observations

k = Number of parameters in the model

L = Maximum value of the likelihood function for the model

Process of Applying AIC and BIC:

1. **Define State Space Models:**
 - Various state space models were specified by adjusting parameters or model complexity.
2. **Fit Models and Calculate Criteria:**
 - For each model, the transition matrix and pattern probabilities were calculated. The log-likelihood for each model was then computed.
 - The AIC and BIC values were derived using the formulas provided.
3. **Model Comparison:**
 - Models were compared based on their AIC and BIC values. The model with the lowest AIC and BIC values was identified as the best fit.

```

def calculate_transition_matrix(data, pattern_length, label):
    pattern_probabilities = calculate_probabilities(data, pattern_length)
    patterns = generate_all_patterns(pattern_length)
    transition_matrix = []
    for pattern in patterns:
        no_rain_prob, rain_prob = pattern_probabilities[tuple(pattern)]
        transition_matrix.append([no_rain_prob, rain_prob])
    transition_df = pd.DataFrame(transition_matrix, index=[tuple(p) for p in patterns], columns=['P(0)', 'P(1)'])

    # Calculate AIC and BIC
    n = len(data)
    k = (2**pattern_length) * (2**pattern_length) - (2**pattern_length)
    log_likelihood = calculate_log_likelihood(pattern_probabilities, data, pattern_length)
    aic = 2 * k - 2 * log_likelihood
    bic = np.log(n) * k - 2 * log_likelihood

    print(f"\n{label} Transition Matrix for Pattern Length {pattern_length}:")
    print(transition_df)
    print(f"AIC: {aic:.2f}")
    print(f"BIC: {bic:.2f}")

    return transition_df, aic, bic

```

calculate_transition_matrix Function: Computes the transition matrix and evaluates the model using AIC and BIC.

For a model where the transition probabilities depend only on the previous month:

- **AIC:** 181.33
- **BIC:** 187.26

For a model where the transition probabilities depend only on the previous year:

- **AIC:** 19.11
- **BIC:** 20.08

For a model where the transition probabilities depend on the previous two months:

- **AIC:** 199.32
- **BIC:** 234.88

For a model where the transition probabilities depend on the previous two years:

- **AIC:** 32.32
- **BIC:** 38.14

For a model where the transition probabilities depend on the previous three months:

- **AIC:** 275.75
- **BIC:** 441.67

For a model where the transition probabilities depend on the previous three years:

- **AIC:** 115.82
- **BIC:** 142.97

For a model where the transition probabilities depend on the previous four months:

- **AIC:** 624.38
- **BIC:** 1335.46

For a model where the transition probabilities depend on the previous four years:

- **AIC:** 482.77
- **BIC:** 599.15

For a model where the transition probabilities depend on the previous five months:

- **AIC:** 2102.65
- **BIC:** 5041.79

For a model where the transition probabilities depend on the previous five years:

- **AIC:** 1986.77
- **BIC:** 2467.80

Analysis

- **Lowest AIC:** The model where transition probabilities depend on the previous year (AIC: 19.11) has the lowest AIC, suggesting it fits the data best among the considered models when accounting for complexity.
- **Lowest BIC:** Similarly, the model where transition probabilities depend on the previous year (BIC: 20.08) has the lowest BIC, indicating it is the best model when penalizing for complexity more strictly.

Conclusion

The model where transition probabilities depend only on the previous year has the best fit based on both AIC and BIC. This model strikes a good balance between fit and complexity, making it the most suitable choice among the models considered.

- **Best Model:** Transition probabilities depend on the previous year.

Part 2:

In this part of the modeling, we are not using SPI; instead, we are modeling based on the mean rainfall for each month and each year.

First, we calculate the average rainfall for each month and each year and save it to an Excel file. Then, we categorize each average value using the function `def categorize_mean_rainfall(mean_rainfall):` and save the results to the Excel file as follows:

```
def categorize_mean_rainfall(mean_rainfall):  
    if 0 <= mean_rainfall < 1:  
        return 0  
    elif 1 <= mean_rainfall < 2:  
        return 1  
    elif 2 <= mean_rainfall < 3:  
        return 2  
    elif 3 <= mean_rainfall < 4:  
        return 3  
    elif 4 <= mean_rainfall < 5:  
        return 4  
    elif 5 <= mean_rainfall < 6:  
        return 5  
    elif 6 <= mean_rainfall < 7:  
        return 6  
    elif 7 <= mean_rainfall < 8:  
        return 7  
    elif 8 <= mean_rainfall < 9:  
        return 8  
    elif 9 <= mean_rainfall < 10:  
        return 9  
    else:  
        return np.nan
```

The table below is for the model for each month:

Date	Mean_Rainfall	Mean_Rainfall_category
01/31/2009	0.322580645	0
02/28/2009	0.353571429	0
03/31/2009	0.103225806	0
04/30/2009	1.103333333	1
05/31/2009	0.44516129	0
06/30/2009	0.703333333	0
07/31/2009	0	0
08/31/2009	0.003225806	0
09/30/2009	0.19	0
10/31/2009	0.14516129	0
11/30/2009	0.666666667	0
12/31/2009	0.329032258	0
01/31/2010	0.109677419	0
02/28/2010	0.403571429	0
03/31/2010	0.45483871	0
04/30/2010	0.376666667	0
05/31/2010	0.503225806	0
06/30/2010	0.073333333	0
07/31/2010	0.009677419	0
08/31/2010	0.690322581	0
09/30/2010	0.743333333	0
10/31/2010	0.387096774	0
11/30/2010	0.296666667	0
12/31/2010	0.093548387	0

01/31/2011	0.329032258	0
02/28/2011	0.360714286	0
03/31/2011	0.906451613	0
04/30/2011	0.42	0
05/31/2011	0.858064516	0
06/30/2011	0.916666667	0
07/31/2011	0.429032258	0
08/31/2011	0.390322581	0
09/30/2011	0.103333333	0
10/31/2011	1.396774194	1
11/30/2011	0.746666667	0
12/31/2011	0.106451613	0
01/31/2012	0.370967742	0
02/29/2012	0.267857143	0
03/31/2012	0.583870968	0
04/30/2012	1.453333333	1
05/31/2012	1.593548387	1
06/30/2012	1.206666667	1
07/31/2012	0.880645161	0
08/31/2012	0.261290323	0
09/30/2012	0.1	0
10/31/2012	0.55	0
11/30/2012	0.216129032	0
12/31/2012	0.277419355	0
01/31/2013	0.2	0
02/28/2013	0.435483871	0
03/31/2013	0.366666667	0
04/30/2013	0.85483871	0
05/31/2013	0.266666667	0
06/30/2013	0.006451613	0
07/31/2013	0.093548387	0
08/31/2013	0.116666667	0
09/30/2013	0.209677419	0
10/31/2013	0.543333333	0
11/30/2013	0.509677419	0
12/31/2013	0.183870968	0
01/31/2014	0.189285714	0
02/28/2014	0.541935484	0
03/31/2014	1.013333333	1
04/30/2014	1.2	1
05/31/2014	0.723333333	0
06/30/2014	0.358064516	0
07/31/2014	0.877419355	0
08/31/2014	0.11	0
09/30/2014	0.209677419	0
10/31/2014	0.313333333	0
11/30/2014	0.309677419	0
12/31/2014	0.209677419	0
01/31/2015	1.285714286	1
02/28/2015	0.816129032	0

03/31/2015	0.84	0
04/30/2015	0.858064516	0
05/31/2015	0.456666667	0
06/30/2015	0.458064516	0
07/31/2015	0.290322581	0
08/31/2015	0.646666667	0
09/30/2015	0.861290323	0
10/31/2015	0.703333333	0
11/30/2015	0.825806452	0
12/31/2015	0.725806452	0
01/31/2016	0.492857143	0
02/29/2016	1.051612903	1
03/31/2016	0.78	0
04/30/2016	0.609677419	0
05/31/2016	0.703333333	0
06/30/2016	0.212903226	0
07/31/2016	0.041935484	0
08/31/2016	0.153333333	0
09/30/2016	0.070967742	0
10/31/2016	0.453333333	0
11/30/2016	1.680645161	1
12/31/2016	0.74516129	0
01/31/2017	0.503571429	0
02/28/2017	0.851612903	0
03/31/2017	1.436666667	1
04/30/2017	0.561290323	0
05/31/2017	0.053333333	0
06/30/2017	0.625806452	0
07/31/2017	0.048387097	0
08/31/2017	0.006666667	0
09/30/2017	0.293548387	0
10/31/2017	0.136666667	0
11/30/2017	0.25483871	0
12/31/2017	0.796774194	0
01/31/2018	1.421428571	1
02/28/2018	0.425806452	0
03/31/2018	1.466666667	1
04/30/2018	1.464516129	1
05/31/2018	0.4	0
06/30/2018	0.003225806	0
07/31/2018	0.022580645	0
08/31/2018	0.266666667	0
09/30/2018	1.125806452	1
10/31/2018	1.006666667	1
11/30/2018	0.512903226	0
12/31/2018	2.774193548	2
01/31/2019	0.953571429	0
02/28/2019	1.332258065	1
03/31/2019	1.72	1
04/30/2019	1.151612903	1

05/31/2019	0.21	0
06/30/2019	0.016129032	0
07/31/2019	0.080645161	0
08/31/2019	0.036666667	0
09/30/2019	1.822580645	1
10/31/2019	2.286666667	2
11/30/2019	2.067741935	2
12/31/2019	0.948387097	0
01/31/2020	5.867857143	5
02/29/2020	3.183870968	3
03/31/2020	4.746666667	4
04/30/2020	1.6	1
05/31/2020	0.146666667	0
06/30/2020	0.290322581	0
07/31/2020	0.225806452	0
08/31/2020	0.043333333	0
09/30/2020	2.106451613	2
10/31/2020	3.071428571	3

The table below is for the model for each year:

Date	Mean_Rainfall	Mean_Rainfall_category
12/31/2009	0.360547945	0
12/31/2010	0.344383562	0
12/31/2011	0.582465753	0
12/31/2012	0.664383562	0
12/31/2013	0.324383562	0
12/31/2014	0.504657534	0
12/31/2015	0.683013699	0
12/31/2016	0.582739726	0
12/31/2017	0.46	0
12/31/2018	0.736712329	0
12/31/2019	1.207945205	1
12/31/2020	1.932075472	1

```

def calculate_log_likelihood_transition_matrix(transition_matrix, df, column_name):
    log_likelihood = 0
    for i in range(1, len(df)):
        prev_category = df.at[i - 1, column_name]
        current_category = df.at[i, column_name]
        if not pd.isna(prev_category) and not pd.isna(current_category):
            prob = transition_matrix.at[prev_category, current_category]
            if prob > 0:
                log_likelihood += np.log(prob)
    return log_likelihood

def calculate_transition_matrix_one_day(df, column_name):
    unique_categories = sorted(df[column_name].dropna().unique())
    num_categories = len(unique_categories)
    transition_matrix = pd.DataFrame(0, index=unique_categories, columns=unique_categories, dtype=float)

    for i in range(1, len(df)):
        prev_category = df.at[i - 1, column_name]
        current_category = df.at[i, column_name]
        if not pd.isna(prev_category) and not pd.isna(current_category):
            transition_matrix.at[prev_category, current_category] += 1

    transition_matrix = transition_matrix.div(transition_matrix.sum(axis=1), axis=0).fillna(0)

    n = len(df) - 1
    k = num_categories * (num_categories - 1)
    log_likelihood = calculate_log_likelihood_transition_matrix(transition_matrix, df, column_name)
    aic = 2 * k - 2 * log_likelihood
    bic = np.log(n) * k - 2 * log_likelihood

    return transition_matrix, aic, bic

```

These two functions calculate the transition matrix for a categorical column, considering transitions from one month to the next. They also compute the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) for model evaluation.

Function 1: calculate_log_likelihood_transition_matrix

- **Purpose:** Computes the log-likelihood of the observed transitions based on the transition matrix.
- **Steps:**
 1. Loop through the DataFrame from the second row.
 2. For each transition, get the probability from the matrix.
 3. Add the log of this probability to the log-likelihood if the probability is greater than zero.

Function 2: calculate_transition_matrix_one_day

- **Purpose:** Calculates the transition matrix and evaluates it using AIC and BIC.
- **Steps:**
 1. **Compute Transition Matrix:** Count transitions from one day to the next and normalize to probabilities.
 2. **Calculate AIC and BIC:**
 - **AIC:** $2 * k - 2 * \log_likelihood$
 - **BIC:** $np.log(n) * k - 2 * \log_likelihood$
 3. **Return:** Transition matrix, AIC, and BIC.

The transition probabilities, AIC and BIC are based on the state from the previous month:

```
Transition Matrix for Mean_Rainfall_category (one day):
      0      1      2      3      4      5
0 0.859649 0.114035 0.017544 0.00 0.0 0.008772
1 0.619048 0.333333 0.047619 0.00 0.0 0.000000
2 0.500000 0.000000 0.250000 0.25 0.0 0.000000
3 0.000000 0.000000 0.000000 0.00 1.0 0.000000
4 0.000000 1.000000 0.000000 0.00 0.0 0.000000
5 0.000000 0.000000 0.000000 1.00 0.0 0.000000
AIC: 213.99
BIC: 302.67
```

The transition probabilities, AIC, and BIC are based on the state from two months prior:

```
      0      1      2      3      4      5
0 0.877551 0.112245 0.010204 0.0 0.0 0.0
1 0.538462 0.384615 0.076923 0.0 0.0 0.0
2 0.500000 0.000000 0.000000 0.5 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 0.0 0.0
4 0.000000 0.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 1.0 0.0 0.0
1 0.846154 0.076923 0.076923 0.0 0.0 0.0
1 0.714286 0.285714 0.000000 0.0 0.0 0.0
2 0.000000 0.000000 1.000000 0.0 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 0.0 0.0
4 0.000000 0.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 0.0 0.0 0.0
2 0.000000 0.500000 0.000000 0.0 0.0 0.5
1 0.000000 0.000000 0.000000 0.0 0.0 0.0
2 1.000000 0.000000 0.000000 0.0 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 0.0 0.0
4 0.000000 0.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 0.0 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 0.0 0.0
1 0.000000 0.000000 0.000000 0.0 0.0 0.0
2 0.000000 0.000000 0.000000 0.0 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 0.0 0.0
4 0.000000 1.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 0.0 0.0 0.0
4 0.000000 0.000000 0.000000 0.0 0.0 0.0
1 1.000000 0.000000 0.000000 0.0 0.0 0.0
2 0.000000 0.000000 0.000000 0.0 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 0.0 0.0
4 0.000000 0.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 0.0 0.0 0.0
1 0.000000 0.000000 0.000000 0.0 0.0 0.0
2 0.000000 0.000000 0.000000 0.0 0.0 0.0
3 0.000000 0.000000 0.000000 0.0 1.0 0.0
4 0.000000 0.000000 0.000000 0.0 0.0 0.0
5 0.000000 0.000000 0.000000 0.0 0.0 0.0
AIC: 490.96
BIC: 1021.74
```

We will continue in the same manner for other cases.

Part 3: Repeat Part, but relax assume (i)

We aim to estimate the parameters of a Hidden Markov Model (HMM) using observed weather data and compare the hidden states generated by the model with actual rain data. Our observed variables include humidity, cloud cover, temperature, pressure, wind, and general weather conditions. The hidden states we are interested in are "rainy" and "sunny".

2. HMM and Its Capabilities

In HMMs, three primary questions are addressed:

1. **Given the model parameters and observed data, estimate the optimal sequence of hidden states.**
2. **Given the model parameters and observed data, calculate the model likelihood.**
3. **Given just the observed data, estimate the model parameters.**

In our case, we will focus on the third question: **estimating the model parameters given just the observed data.**

3. Data Preparation

3.1 Compute Monthly Averages

- **Observation Data:** For each observation variable (humidity, cloud cover, temperature, pressure, wind, and weather), calculate the mean for each month. This will give us a vector of mean values for each month.
- **Rain Data:** Similarly, compute the mean rain value for each month.

3.2 Organize Data

- Create a matrix where each row represents a month and each column contains the mean values of the corresponding observation variables. The resulting matrix will have months as rows and observation types as columns.

4. Model Assumptions

- We assume that the observation data follows a normal distribution.

5. Standardize the Data

- Standardize the observation data to ensure each feature has a mean of zero and a standard deviation of one. This normalization step is crucial for improving the performance of the HMM.

6. Fit the HMM

- **Calculate Statistics:** Compute the mean and covariance matrix of the standardized data.
- **Model Training:** Use the hmmlearn library in Python to fit the HMM to the standardized data. This involves:
 - Initializing the HMM with the calculated statistics.
 - Training the HMM on the observation data to estimate the model parameters.

7. Convert Rain Data to States

- Define a threshold to classify the mean rain data into two states: "sunny" or "rainy".
 - **Threshold Calculation:** Use the median of the monthly mean rain values as the threshold.
 - **State Assignment:** Classify months with mean rain values above the threshold as "rainy" and those below as "sunny".

8. Compare Hidden States with Rain Data

- After fitting the HMM, most likely sequence of hidden states.
- Compare these hidden states with the classified rain states ("sunny" or "rainy") to evaluate the model's performance.

Year	Month	Max	Min	Wind	Rain	Humidity	Cloud	Pressure	Weather_Code
2009	1	4.548387	-5.06452	7.16129	0.322581	55.58065	13.3871	1021.645	1.928571429
2009	2	7.714286	-0.46429	9.392857	0.353571	57.14286	15.71429	1016.286	1.888888889
2009	3	13.41935	2.709677	10.3871	0.103226	37.32258	14.3871	1013.258	1.903225806
2009	4	13.96667	4.366667	8.666667	1.103333	50.66667	13.83333	1015.067	1.896551724
2009	5	23.96774	12.35484	9.322581	0.445161	35.06452	11.6129	1013.258	1.933333333
2009	6	28.46667	15.9	8.333333	0.703333	32.7	7.833333	1013.067	2
2009	7	35.06452	21.19355	12.12903	0	18.41935	2.806452	1005.677	2
2009	8	33.87097	19.77419	10.6129	0.003226	25.67742	4.580645	1009.065	2
2009	9	27.23333	15.23333	9.033333	0.19	30.56667	3.8	1012.533	2
2009	10	21.06452	9.225806	6.967742	0.145161	33	6.258065	1018.613	1.967741935
2009	11	11.96667	3.733333	7.933333	0.666667	53.76667	14.26667	1018.933	1.928571429
2009	12	6.580645	-1	6.580645	0.329032	65.48387	16.09677	1019.71	1.866666667
2010	1	7.870968	-0.12903	8.354839	0.109677	52.29032	15.3871	1020	1.8
2010	2	8.428571	0.285714	9.321429	0.403571	59.14286	19.32143	1016.393	1.846153846
2010	3	14.70968	4.774194	10.03226	0.454839	49.90323	14.90323	1016.968	1.838709677
2010	4	19.06667	7.666667	9.633333	0.376667	43.9	11.23333	1015.067	1.964285714
2010	5	24.80645	12.06452	9.967742	0.503226	35.09677	9.225806	1011.419	1.935483871
2010	6	32.73333	18.3	9.3	0.073333	19.76667	3.7	1008.567	2
2010	7	36.16129	21.74194	10.09677	0.009677	19.48387	2.032258	1006.677	2
2010	8	31.06452	18.51613	6.354839	0.690323	24.90323	7.967742	1011.194	1.935483871
2010	9	27.9	14.76667	5.933333	0.743333	28.5	3.566667	1014.067	2
2010	10	23.29032	11	5.774194	0.387097	26.3871	5.193548	1015.387	2
2010	11	14.1	3.1	4.4	0.296667	38.3	6.6	1020.967	2
2010	12	10.12903	-1.19355	4.741935	0.093548	36.35484	7.129032	1020.161	1.966666667
2011	1	3.612903	-6.64516	5.645161	0.329032	59.77419	18.90323	1021.484	1.793103448
2011	2	5.678571	-5.17857	7.464286	0.360714	59.32143	18.82143	1015.321	1.846153846
2011	3	11.22581	0.709677	6.419355	0.906452	51	23.22581	1018	1.826086957
2011	4	18.56667	6.733333	7.566667	0.42	34.96667	10.13333	1013.533	1.933333333
2011	5	24.16129	14.29032	8.774194	0.858065	28.29032	12.51613	1011.871	1.928571429
2011	6	30.23333	20.6	9.7	0.916667	17.56667	5.1	1007.667	2
2011	7	32.41935	24	8.612903	0.429032	14.58065	9.129032	1007.161	1.967741935

2011	8	31.25806	22.77419	8.741935	0.390323	19.64516	7.129032	1008.032	1.966666667
2011	9	26.06667	17.43333	7.166667	0.103333	18.56667	2.5	1012.2	2
2011	10	18.77419	10.90323	7.709677	1.396774	31.16129	13.16129	1016.484	1.931034483
2011	11	7.066667	0	5.5	0.746667	51.13333	18.53333	1020.4	1.892857143
2011	12	4.709677	-3.77419	4.83871	0.106452	49.67742	10	1024.484	1.935483871
2012	1	3.612903	-4.83871	6.612903	0.370968	48.48387	15.12903	1019.226	1.928571429
2012	2	2.535714	-5.35714	7.357143	0.267857	54.17857	19.78571	1021.357	1.692307692
2012	3	8.096774	-0.83871	9.322581	0.583871	40.54839	13	1015.935	2
2012	4	17.83333	9.333333	8	1.453333	40.56667	10.66667	1013	1.964285714
2012	5	23.96774	14.80645	9.580645	1.593548	28	6.612903	1011.516	2
2012	6	28.56667	19.73333	7.466667	1.206667	22.93333	6.7	1010.133	2
2012	7	31.06452	22.54839	8.064516	0.880645	20.29032	5.516129	1007.839	2
2012	8	31.3871	23.03226	7.548387	0.26129	16.3871	5.935484	1009.097	2
2012	9	26.56667	18.13333	6.733333	0.1	21.06667	4.233333	1013.333	1.966666667
2012	10	20.06452	11.87097	5.580645	0.470968	28.12903	9.16129	1018.452	1.903225806
2012	11	12.3	5.633333	5.233333	0.55	46.46667	14.9	1020.233	1.821428571
2012	12	6.354839	-0.48387	5.16129	0.216129	44.64516	15.48387	1020.323	1.933333333
2013	1	5.419355	-2.67742	7.064516	0.277419	44.64516	12.54839	1018.871	1.966666667
2013	2	8.607143	0.392857	6.642857	0.2	39.75	12.28571	1018.036	1.892857143
2013	3	13	4.645161	7.193548	0.435484	36.96774	16.90323	1014.226	1.793103448
2013	4	18.13333	10.1	6.766667	0.366667	27.23333	12.4	1013.4	1.866666667
2013	5	22.70968	14.19355	8.903226	0.854839	28.54839	11.06452	1012.742	1.935483871
2013	6	29.66667	19.3	9.766667	0.266667	18.66667	2.233333	1006.9	2
2013	7	33.6129	23.41935	9.193548	0.006452	16.29032	0.870968	1005.226	2
2013	8	30.90323	22.19355	8.612903	0.093548	20.35484	3.870968	1008.387	2
2013	9	27.86667	19.13333	6.966667	0.116667	11.43333	1.766667	1009.967	2
2013	10	19.6129	10.51613	6.032258	0.209677	24.58065	3.193548	1017.419	2
2013	11	11.63333	4.933333	4.533333	0.543333	47.06667	20.63333	1020.1	1.740740741
2013	12	5.225806	-1.74194	6.193548	0.509677	50.3871	16.12903	1023.516	1.962962963
2014	1	4.903226	-2.58065	5.612903	0.183871	55.06452	16.67742	1022.29	1.833333333
2014	2	4.892857	-4.10714	5.928571	0.189286	52.92857	15.5	1022.964	1.92
2014	3	12.06452	3.870968	8.193548	0.541935	42.58065	17.3871	1015.323	1.892857143

2014	4	18.33333	9.4	8.166667	1.013333	35.23333	12.6	1014.4	1.928571429
2014	5	24.77419	15.77419	7.935484	1.2	26.74194	10.51613	1012.258	1.965517241
2014	6	29.76667	20.26667	9.266667	0.723333	18.56667	6.666667	1008.933	2
2014	7	33.12903	24.3871	9.193548	0.358065	16.64516	10.45161	1007.387	1.965517241
2014	8	32.3871	23.58065	7.83871	0.877419	16.25806	5.096774	1008.935	2
2014	9	28.4	19.53333	7.5	0.11	13.2	3.7	1010.733	1.965517241
2014	10	19.19355	10.87097	7.16129	0.209677	32.45161	7.903226	1018.419	1.967741935
2014	11	9.8	2.8	5.233333	0.313333	42.33333	14.73333	1020.8	1.896551724
2014	12	7.064516	-0.51613	5.258065	0.309677	50.45161	13.35484	1022.613	1.935483871
2015	1	6.032258	-2	5.645161	0.209677	43.3871	17.12903	1020.613	1.866666667
2015	2	7.392857	-0.35714	6.535714	1.285714	54.92857	22.35714	1016.5	1.84
2015	3	11.12903	2.645161	8.16129	0.816129	48.80645	32.70968	1016.903	1.789473684
2015	4	18.6	9.733333	9.266667	0.84	33.46667	15.86667	1013.467	2
2015	5	25.32258	15.87097	9.096774	0.858065	25.09677	10.03226	1011.581	1.903225806
2015	6	31.56667	22.2	9.4	0.456667	20.43333	6.333333	1008.333	1.966666667
2015	7	32	23.6129	7.354839	0.458065	21.54839	7.806452	1009.194	2
2015	8	30.93548	22.70968	6.258065	0.290323	22.64516	1.935484	1011.355	2
2015	9	25.8	18.23333	5.666667	0.646667	27.96667	13.4	1014.767	2
2015	10	20.70968	13.35484	7.193548	0.86129	39.64516	15.90323	1017.355	1.928571429
2015	11	10.4	3.833333	5.466667	0.703333	48.23333	19.96667	1019.6	1.857142857
2015	12	5.064516	-2.3871	5.548387	0.825806	57.93548	23.41935	1024.387	1.814814815
2016	1	6.096774	-2.54839	6.129032	0.725806	53.77419	19.70968	1018.484	1.962962963
2016	2	8.464286	-1.17857	6.178571	0.492857	49.21429	15.82143	1021.643	1.96
2016	3	12.83871	4.290323	9.096774	1.051613	45.41935	19.45161	1013.871	1.892857143
2016	4	17.73333	8.733333	7.666667	0.78	39.76667	23.7	1014.233	1.952380952
2016	5	24.51613	15.35484	9.612903	0.609677	31.3871	11.35484	1010.968	2
2016	6	28.53333	18.6	10.63333	0.703333	24.43333	9.2	1010.467	1.962962963
2016	7	32.48387	22.6129	9.322581	0.212903	24.22581	19.87097	1007.419	2
2016	8	30.35484	20.80645	5.645161	0.041935	21.16129	2.548387	1011.903	2
2016	9	27.86667	18.26667	7.8	0.153333	22.83333	3.866667	1011.3	2
2016	10	19.54839	10.32258	5.516129	0.070968	33.03226	8.903226	1018.387	1.933333333
2016	11	10.73333	2.866667	5.4	0.453333	37.26667	10.56667	1023.667	2

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from hmmlearn import hmm
import pandas as pd

df = pd.read_excel('/content/data_new.xlsx', index_col=0)
# display(df)

df['Weather_Code'] = df['Weather_Code'].replace(np.nan, 0)
df = df.replace(np.nan, 0)
df.replace(np.nan, 0, inplace=True)
df_x = df_new.drop('Rain',axis = 1)

model = hmm.GaussianHMM(n_components=6, covariance_type='full', n_iter=1000)

# Fit the model to the data
model.fit(df_x_matrix)

# Calculate the emission probabilities for the sample data
emission_probs = model.predict_proba(df_x_matrix)

# Display the first few emission probabilities
print(emission_probs[:6])

# Define the state space
states = ["Rainy"]
n_states = len(states)
print('Number of hidden states :',n_states)
# Define the observation space
observations = ['Min', 'Wind', 'Humidity', 'Cloud', 'Pressure', 'Weather_Code']
n_observations = len(observations)
print('Number of observations :',n_observations)

# Define the initial state distribution
state_probability = np.array([[0.1, 0.2, 0.2, 0.1, 0.2, 0.2],[0.2, 0.3, 0.1, 0.1, 0.1, 0.2]])
print("State probability: ", state_probability)

# Define the state transition probabilities
transition_probability = np.array([[0.859649, 0.114035, 0.017544, 0.00, 0.0, 0.008772],
[0.619048, 0.333333, 0.047619, 0.00, 0.0, 0.000000],
[0.500000, 0.000000, 0.250000, 0.25, 0.0, 0.000000],
[0.000000, 0.000000, 0.000000, 0.00, 1.0, 0.000000],
[0.000000, 1.000000, 0.000000, 0.00, 0.0, 0.000000],
[0.000000, 0.000000, 0.000000, 1.00, 0.0, 0.000000]])
print("\nTransition probability:\n", transition_probability)
# Define the observation likelihoods
emission_probability= emission_probs[:6]
print("\nEmission probability:\n", emission_probability)

model = hmm.CategoricalHMM(n_components=6)
model.startprob_ = state_probability
model.transmat_ = transition_probability
model.emissionprob_ = emission_probability

rainy_state = (df['Rain'] > 0).astype(int)

features = df[['Max', 'Min', 'Wind', 'Humidity', 'Cloud', 'Pressure']]
target = df['Rain']

# Step 5: Train the HMM
# Initialize the HMM
model = hmm.GaussianHMM(n_components=2, covariance_type="diag", n_iter=1000)

# Fit the model
model.fit(features)

# Step 6: Predict the sequence
# Predict the hidden states
hidden_states = model.predict(features)

# Step 7: Map the hidden states to 0 and 1 based on the rainy state
# Assuming 'Rain' > 0 indicates a rainy state
rainy_state = (df['Rain'] > 0).astype(int)

# Map the hidden states to 0 and 1
state_mapping = {0: 0, 1: 1} # Adjust this mapping based on the model's output
predicted_sequence = [state_mapping[state] for state in hidden_states]

# Print the predicted sequence
print(predicted_sequence)
print("Most likely hidden states:", hidden_states)

```

```

[0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0,
0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0]

```